Agentic ATS: Project Proposal

Authors: Rafiq, Abdullah; Macha, Sairamya; Velez, Noah

The Problem

Recruiters miss qualified candidates because keyword screening rewards the right words, not the right experience. A candidate who "led a five-person analytics team" can be filtered out of a role asking for "people management," while a resume that repeats the job ad's keywords sails through.

We will build an agentic screening assistant that reads a job description and a pool of resumes, decides what the role actually requires, finds candidates whose experience matches it even when the wording differs, and hands the recruiter a shortlist where every claim cites the resume line it came from. The recruiter makes the final call — nothing is ever sent to a candidate.

Why an Agent, and Why More Than One

Scoring every resume against a fixed checklist is a script. We put agents only where the next step depends on what was just found:

- **Interpreting the job description** — turning vague requirements ("strong communication skills") into must-haves, nice-to-haves, and acceptable equivalents, and deciding when to ask the recruiter for clarification.
- **Deciding when the search is good enough** — broadening or tightening queries based on how many strong matches come back, rather than running a fixed number of searches.
- **Weighing evidence that isn't a keyword** — transferable experience, career changes, or a keyword list with nothing behind it all require reading, not matching.
- **Checking its own claims** — every "meets the requirement" must point at a resume line; unsupported claims go back for re-evaluation.

These are separate jobs with separate inputs, so they're separate agents: one agent holding the job ad, search results, and 20 full resumes at once would be slower, costlier, and harder to check. We'll prove this rather than claim it, by measuring the agents against a keyword-matching baseline and an embedding-similarity baseline. If the agents don't beat them, we'll report that honestly.

Design: Four Agents, Two Loops, One Human Decision

Agent	Decides	Hands Off
Requirements	Must-haves, nice-to-haves, equivalents; whether to ask the recruiter first	Weighted requirement list
Search	Which queries to run, and whether to broaden, tighten, or stop	Candidate pool (~10–20)
Evaluator	Whether each requirement is met, partly met, or missing, with evidence	Scores with cited resume lines
Verifier	Whether each claim is supported by the line it cites	Pass, or claims sent back to the Evaluator

A thin orchestrator passes results between agents and caps every loop so a bug can't run up the API bill. Two loops send work backward: search repeats until the candidate pool is good enough, and verification repeats until every claim is supported. The recruiter reviews the final shortlist with citations and approves, rejects, or requests a closer look.

Enrichment: RAG over Resumes, with a Human in the Loop

RAG is our primary enrichment because the resume pool is too large to hand the model at once, and we can name exactly what a wrong retrieval costs:

- **A qualified candidate never comes back from search** — the worst failure, because it's silent. We catch it by measuring retrieval recall separately: for each job description, we plant candidates whose correct verdict we already know, and check how many appear in the top-k search results before any model writes anything.
- **A chunk from the wrong candidate gets cited** — our tools only return chunks for the candidate being evaluated, and the Verifier rejects any citation whose candidate ID doesn't match.
- **A claim isn't actually in the resume** — the Evaluator must quote the resume line behind each claim, and code checks the quote appears verbatim, no second model call required.

Human-in-the-loop is our second enrichment: the recruiter answers the Requirements agent's clarifying questions and makes the final call on every shortlist.

Data and Evaluation

We generate synthetic resumes and job descriptions ourselves (no real candidate data in the public repo), planting known candidate types — clear fits, "hidden gems" described in different words, keyword stuffers, near misses, and career changers — so we have an answer key instead of eyeballing results.

Three systems face the same job descriptions: a keyword-matching baseline, an embedding-similarity baseline, and our agentic system. We measure retrieval recall, hidden-gem recall, keyword-stuffer rejection, near-miss accuracy, unsupported-claim rate, and cost/time per job — so a failure points at the specific stage that caused it. Every run saves its full trace for debugging.

Risks

Risk	Mitigation
Bias against groups of candidates	Evaluator sees only experience, education, and skills — no names, photos, ages, or addresses
Confident but unsupported claims	Every claim must quote the resume; code verifies the quote exists
A qualified candidate silently missed	Retrieval recall measured separately against planted fits
Agents no better than simple rules	Both baselines are built first; we report the comparison either way
API cost runs away	Every loop is capped; cheapest viable model, chosen by measuring

This is a class project on synthetic data, not a tool for real hiring decisions — the README will say so.

Plan: Scope and Ownership

We build a small end-to-end version first, then widen it if time allows.

Minimum (must ship): 1 job description, ~50 resumes, all four agents with capped loops and saved traces, both baselines with metrics, a simple Streamlit UI showing the shortlist with citations, hosted publicly with a live URL, and an incident log in the repo tracking at least four deliberately induced failures.

Stretch (only if the minimum works): 3–5 job descriptions and ~150 resumes, recruiter feedback that re-ranks the shortlist, suggested interview questions per gap, and a polished UI with a live trace panel.

Ownership (each of us also reviews the others' work):

- **Abdullah** — Evaluation: planted ground truth, baselines, metrics, traces; the Verifier.
- **Noah** — Data and RAG: resume/job generator, chunking, embeddings, vector store, search tools.
- **Sairamya** — Agents and UI: Requirements, Search, and Evaluator agents, the orchestrator, the app.

Roles are a starting proposal the group can adjust. All code lives in our team's GitHub repo, with API keys kept in `.env` and `.gitignore`'d before the first push.

The Close

If the agents don't beat keyword matching on hidden gems, we'll say so and show where they failed. Our mini-projects taught us that measuring against the simpler option is the point.